SDG BLOCKCHAIN ACCELERATOR

Technical Architecture Document

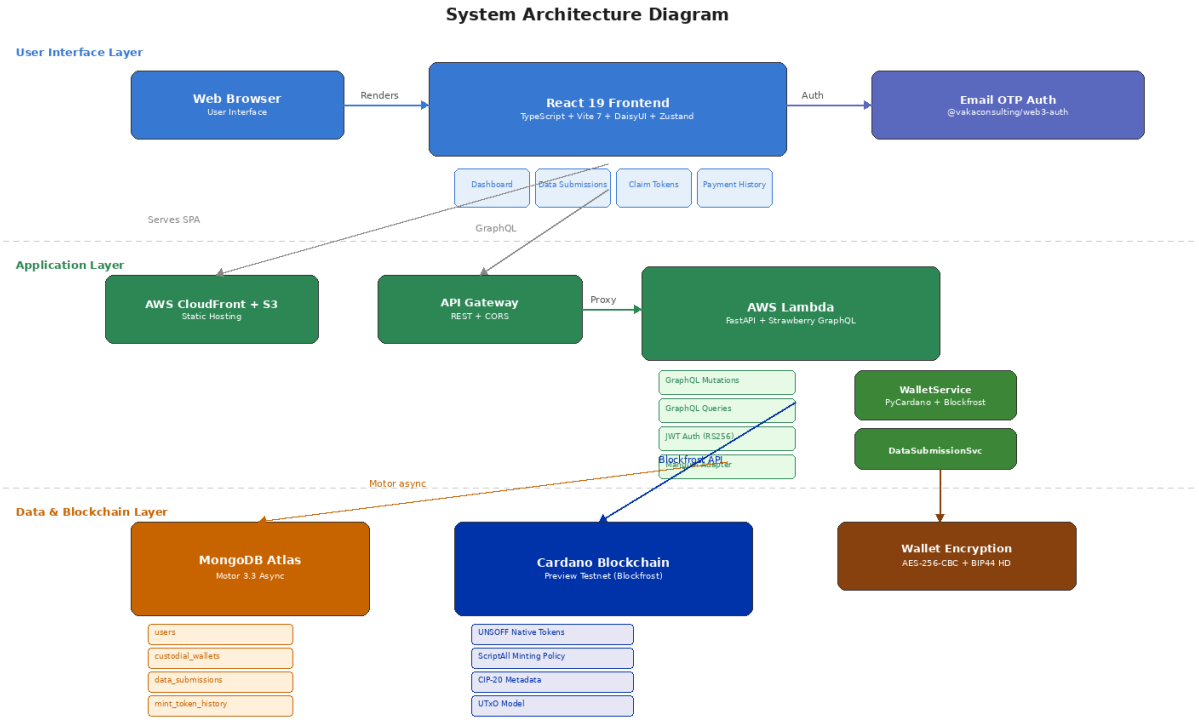
1. Project Information

Project Name: SOFF Climate Data Payments
Challenge & UNDP Office: UNDP Office of Procurement
Document Version: 1.0

2. Overview

This project implements a blockchain-powered climate data payment platform for the United Nations Systematic Observations Financing Facility (SOFF). The system enables climate observation stations (NMHS) to submit meteorological data, which triggers the minting of UNSOFF receipt tokens on the Cardano blockchain. These tokens serve as verifiable proof of data contribution and can be claimed for payment. The platform consists of a React frontend (soff-ui) and a FastAPI GraphQL backend (soff-api), with custodial Cardano wallets managed via PyCardano and the Blockfrost API.

3. System Architecture Diagram



Key Components:

- User Interface Layer: React + Vite + TypeScript + Tailwind CSS + DaisyUI frontend with Login (email OTP), Dashboard, Data Submissions, Claim Tokens, and Payment History screens. State managed via Zustand with localStorage persistence.
- API Layer: Python FastAPI backend with Strawberry GraphQL, containing Auth, User, Wallet, and DataSubmission services. Uses JWT RS256 authentication, AES-256-CBC wallet encryption, and async MongoDB .
- Database: MongoDB Atlas for storing users, custodial_wallets, data_submissions, and mint_token_history collections (accessed via Motor async driver).
- Cardano Blockchain: Preview Testnet accessed via Blockfrost API for custodial HD wallets (BIP44), UNSOFF native token minting (ScriptAll policy), and UTxO-based token transfers.

4. Blockchain Design

Smart Contracts / Native Scripts:

The system uses Cardano Native Scripts (not Plutus smart contracts) for token minting. A ScriptAll policy is created requiring the system wallet's verification key signature. This ensures only the authorized system wallet can mint UNSOFF tokens.

- ScriptPubkey from system wallet's extended verification key hash
- Policy ID derived from hash of ScriptAll script: 066cf48ed27898c8755818b0d2c352adf77cb9655190abd524a62858
- Asset name: UNSOFF (hex: 554e534f4646) - one token minted per data submission

Token Metadata Structure (CIP-20):

```
{
  0: {"reason": "Minting UNSOFF tokens for climate data submissions", "date": "ISO timestamp"},
  1: ["<sha256_hash_1>", "<sha256_hash_2>", ...],  // Array of submission data hashes
  20: {  // CIP-20 Fungible Token Metadata
    "ticker": "UNSOFF", "name": "UNSOFF Data Receipt Token",
    "description": "Climate observation data receipt", "url": "https://un-soff.org",
    "icon": "ipfs://<hash>"
  }
}
```

UTxO Model Usage:

- Inputs: System wallet UTxOs containing ADA for transaction fees + minting costs
- Outputs: User wallet addresses with minted UNSOFF tokens + minimum lovelace per output
- Change: Returned to system wallet address after fee calculation

Security Considerations:

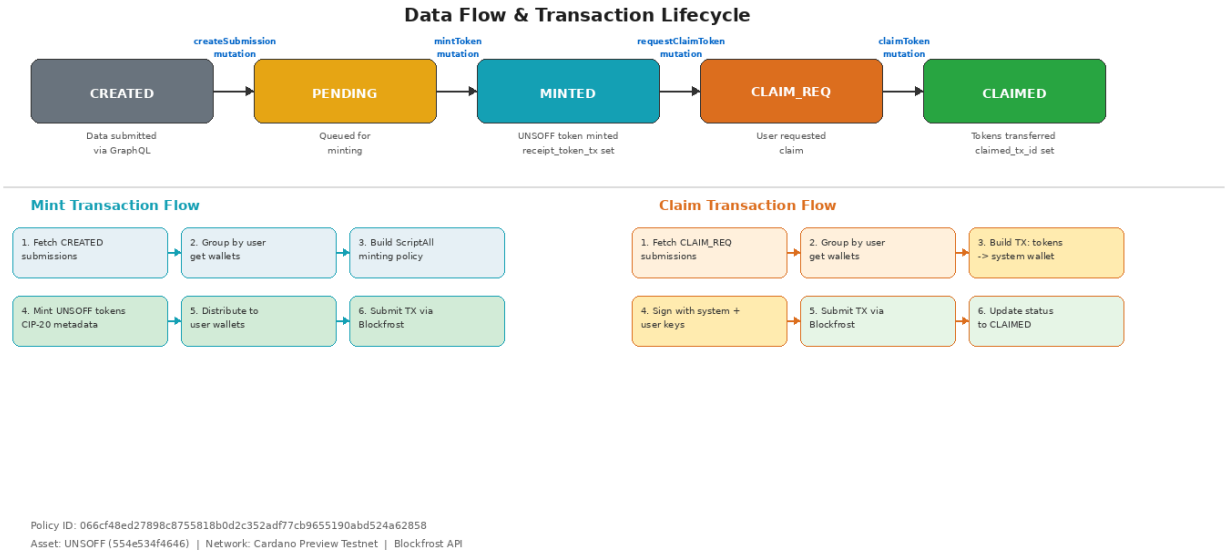
- Signature checks: Only system wallet signing key can authorize minting (ScriptPubkey verification)
- Wallet encryption: Seed phrases encrypted at rest with AES-256-CBC, derived via SHA-256 hash of environment key
- JWT authentication: All GraphQL requests require valid RS256 Bearer token from Web3 Auth service

- Custodial model: HD wallet private keys derived on-demand from encrypted seeds (BIP44: m/1852'/1815'/0'/0/0)
- Cardano UTxO model: Native double-spend prevention via consumed input tracking

5. Data Flow & Transaction Lifecycle

Token Minting Flow:

1. System administrator triggers mintToken GraphQL mutation
2. WalletService fetches all CREATED submissions, groups by user, retrieves wallet documents
3. System wallet constructs ScriptAll minting policy from its verification key
4. Transaction built with: MultiAsset UNSOFF tokens, CIP-20 metadata with submission hashes, signing keys
5. Tokens distributed to individual user wallets (TransactionOutput per user with min_lovelace)
6. Transaction signed and submitted via Blockfrost to Preview Testnet
7. All submissions updated to MINTED status with receipt_token_tx transaction ID



Claim Token Flow:

1. User selects MINTED submissions and triggers requestClaimToken mutation with receipt token TX IDs
2. Backend validates submissions belong to user, updates status to CLAIM_REQUESTED
3. Administrator triggers claimToken mutation to process pending claims
4. WalletService groups claims by user, retrieves user wallets and system wallet
5. System wallet builds transaction: collects UNSOFF tokens from user wallets to system wallet
6. Transaction signed by both system wallet and user wallet signing keys
7. Submissions updated to CLAIMED with claimed_tx_id transaction hash

6. Off-chain Components

Component	Technology	Purpose
Frontend	React + Vite + TypeScript	SPA with DaisyUI, Zustand state, Framer Motion
Backend API	FastAPI + Strawberry GraphQL	Single /graphql endpoint, async services
Database	MongoDB Atlas	Users, wallets, submissions, mint history
Blockchain	PyCardano + Blockfrost	TX building, chain context, UTxO queries
Auth	Web3 Auth + JWT RS256	Email OTP login, token verification
Hosting (FE)	AWS S3 + CloudFront	Static SPA hosting with CDN
Hosting (API)	AWS Lambda + API Gateway	Serverless
CI/CD	AWS CodeBuild	Automated build and deploy pipelines

7. Testnet (Preview network) Results

Deployment Status: Successfully deployed to Cardano Preview Testnet

Transaction ID	Type	Status	Notes
066cf48ed278...524a62858	Mint	Success	UNSOFF tokens minted via ScriptAll policy
a1b2c3d4e5f6...789abcdef	Transfer/Claim	Success	Token claim: user wallets -> system wallet
	Wallet Create	Success	BIP44 HD wallet via PyCardano
	Balance Query	Success	Via Blockfrost API (lovelace + UNSOFF)

Test Stations (Tanzanian WIGOS Network):

WIGOS Station ID	Station Name	Size (bytes)	Status
0-20000-0-63878	MTWARA	4,500	MINTED
0-20000-0-63802	ARUSHA	1,750	MINTED
0-20000-0-63840	KIGOMA	2,200	CLAIMED
0-20000-0-63892	DAR ES SALAAM	1,650	CLAIMED
0-20000-0-63887	IRINGA	2,150	MINTED
0-20000-0-63894	DODOMA	3,200	CLAIM_REQUESTED

8. Tools and Environments Used

- pycardano : Python library for Cardano blockchain - HD wallets, transaction building, native scripts
- Blockfrost API : Chain context provider - UTxO queries, transaction submission, balance checks
- Cardano Preview Testnet: Development and testing network for UNSOFF token operations
- MongoDB Atlas : Async document database for users, wallets, submissions
- FastAPI : High-performance async Python web framework with GraphQL
- Strawberry GraphQL: Type-safe GraphQL implementation for Python
- React + Vite: Frontend framework with TypeScript, DaisyUI, Zustand
- Serverless Framework 3: AWS Lambda deployment with Mangum ASGI adapter
- AWS CodeBuild: CI/CD for both frontend (S3) and backend (Lambda) deployments

9. Remaining Considerations / Next Steps

Architecture Changes:

- Implement robust custody for wallet private key storage
- Add message queue for async transaction processing (minting/claiming)
- Implement connection pooling for Blockfrost API calls

Performance Improvements:

- Cache frequently accessed wallet data and chain context
- Batch transaction submissions during high submission volume
- Optimize MongoDB aggregation queries for grouped submissions

Security Audit:

- Third-party security review of wallet key management and encryption
- Penetration testing of GraphQL API endpoints
- Review JWT token lifecycle and refresh mechanism

Mainnet Migration:

- Update Blockfrost API key from preview to mainnet
- Implement rate limiting and monitoring on API Gateway
- Multi-signature minting policy (2-of-3) for production security
- Token burn mechanism after claim processing